# Technical Requirement Document

---

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

## TR-DLT-001

**Related Functional Requirement(s): FR-DLT-001**

**Objective: Implement a PySpark job to load customer and order data from CSV files into Delta tables.**

### Target Cluster Configuration:

- Cluster Name: Data Engineering Cluster
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

### Source Data Details:

- Customer data CSV file: `/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata` in CSV format
- Order data CSV file: `/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata` in CSV format

### Target Data Details:

- Customer Delta table: `customer_delta` in Delta format
- Order Delta table: `order_delta` in Delta format

### Job Flow / Pipeline Stages:

- Read customer data from CSV file
- Load customer data into Delta table
- Read order data from CSV file
- Load order data into Delta table

### Data Transformations / Business Logic:

- Step 1: Read customer data from CSV file and load into Delta table with schema: `CustId`, `Name`, `EmailId`, `Region`

• Step 2: Read order data from CSV file and load into Delta table with schema: `OrderId`, `ItemName`, `PricePerUnit`, `Qty`, `Date`, `CustId`

**Error Handling and Logging:**

• Log errors during data loading into Delta tables

• Implement retry mechanism for failed data loading

## TR-DLT-002

**Related Functional Requirement(s): FR-DLT-002**

**Objective: Implement a PySpark job to add a new column "TotalAmount" to the order Delta table.**

**Target Cluster Configuration: Same as TR-DLT-001**

**Source Data Details:**

• Order Delta table: `order_delta` in Delta format

**Target Data Details:**

• Updated Order Delta table: `order_delta` in Delta format

**Job Flow / Pipeline Stages:**

• Add new column "TotalAmount" to order Delta table

**Data Transformations / Business Logic:**

• Step 1: Calculate "TotalAmount" as product of `PricePerUnit` and `Qty` columns

• Step 2: Add "TotalAmount" column to order Delta table

**Error Handling and Logging:**

• Log errors during data transformation

• Implement retry mechanism for failed data transformation

## TR-DLT-003

**Related Functional Requirement(s): FR-DLT-003**

**Objective: Implement a PySpark job to remove null and duplicate records from customer and order Delta tables.**

**Target Cluster Configuration: Same as TR-DLT-001**

**Source Data Details:**

• Customer Delta table: `customer_delta` in Delta format

• Order Delta table: `order_delta` in Delta format

**Target Data Details:**

- Cleansed Customer Delta table: `customer_delta` in Delta format
- Cleansed Order Delta table: `order_delta` in Delta format

**Job Flow / Pipeline Stages:**

- Remove null records from customer Delta table
- Remove duplicate records from customer Delta table
- Remove null records from order Delta table
- Remove duplicate records from order Delta table

**Data Transformations / Business Logic:**

- Step 1: Remove null records from customer and order Delta tables
- Step 2: Remove duplicate records from customer and order Delta tables

**Error Handling and Logging:**

- Log errors during data cleansing
- Implement retry mechanism for failed data cleansing

## TR-DLT-004

**Related Functional Requirement(s): FR-DLT-004**

**Objective: Implement a PySpark job to create "ordersummary" table and load joined data from customer and order Delta tables.**

**Target Cluster Configuration: Same as TR-DLT-001**

**Source Data Details:**

- Customer Delta table: `customer_delta` in Delta format
- Order Delta table: `order_delta` in Delta format

**Target Data Details:**

- "ordersummary" table: `ordersummary` in Delta format

**Job Flow / Pipeline Stages:**

- Create "ordersummary" table if it does not exist
- Join customer and order Delta tables using `CustId` as join key
- Load joined data into "ordersummary" table as SCD Type 2 table

**Data Transformations / Business Logic:**

- Step 1: Join customer and order Delta tables using `CustId` as join key
- Step 2: Load joined data into "ordersummary" table with schema: `CustId`, `Name`, `EmailId`, `Region`, `OrderId`, `ItemName`, `PricePerUnit`, `Qty`, `Date`

**Error Handling and Logging:**

- Log errors during data loading into "ordersummary" table

- Implement retry mechanism for failed data loading

## TR-DLT-005

**Related Functional Requirement(s): FR-DLT-005**

**Objective: Implement a PySpark job to update "ordersummary" table on customer data change.**

**Target Cluster Configuration: Same as TR-DLT-001**

**Source Data Details:**

- Customer Delta table: `customer_delta` in Delta format
- "ordersummary" table: `ordersummary` in Delta format

**Target Data Details:**

- Updated "ordersummary" table: `ordersummary` in Delta format

**Job Flow / Pipeline Stages:**

- Identify changed customer records
- Update "ordersummary" table by making old records inactive and new records active
- Update `StartDate` and `EndDate` columns accordingly to maintain history

**Data Transformations / Business Logic:**

- Step 1: Identify changed customer records
- Step 2: Update "ordersummary" table accordingly

**Error Handling and Logging:**

- Log errors during data update
- Implement retry mechanism for failed data update

## TR-DLT-006

**Related Functional Requirement(s): FR-DLT-006**

**Objective: Implement a PySpark job to create "customeraggregatespend" table and load aggregated data from "ordersummary" table.**

**Target Cluster Configuration: Same as TR-DLT-001**

**Source Data Details:**

- "ordersummary" table: `ordersummary` in Delta format

**Target Data Details:**

- "customeraggregatespend" table: `customeraggregatespend` in Delta format

**Job Flow / Pipeline Stages:**

• Create "customeraggregatespend" table if it does not exist

• Aggregate `TotalAmount` column from "ordersummary" table by `Name` and `Date` columns

• Load aggregated data into "customeraggregatespend" table

**Data Transformations / Business Logic:**

• Step 1: Aggregate `TotalAmount` column from "ordersummary" table by `Name` and `Date` columns

• Step 2: Load aggregated data into "customeraggregatespend" table with schema: `Name`, `TotalAmount`, `Date`

**Error Handling and Logging:**

• Log errors during data aggregation and loading

• Implement retry mechanism for failed data aggregation and loading

## TR-DLT-007

**Related Functional Requirement(s): FR-DLT-007**

**Objective: Implement a Delta Live Tables (DLT) pipeline to perform data ingestion, transformation, and loading into Delta tables.**

**Target Cluster Configuration: Same as TR-DLT-001**

**Source Data Details:**

• Customer data CSV file: `/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata` in CSV format

• Order data CSV file: `/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata` in CSV format

**Target Data Details:**

• Various Delta tables: `customer_delta`, `order_delta`, `ordersummary`, `customeraggregatespend` in Delta format

**Job Flow / Pipeline Stages:**

• Create a DLT pipeline to perform data ingestion, transformation, and loading into Delta tables

• Use DLT to implement SCD Type 2 logic for "ordersummary" table

• Use DLT to aggregate data and load into "customeraggregatespend" table

**Data Transformations / Business Logic:**

• Step 1: Implement data ingestion, transformation, and loading into Delta tables using DLT

• Step 2: Implement SCD Type 2 logic for "ordersummary" table using DLT

• Step 3: Aggregate data and load into "customeraggregatespend" table using DLT

**Error Handling and Logging:**

• Log errors during DLT pipeline execution

• Implement retry mechanism for failed DLT pipeline execution